

OBSERVER 2.0

Documento descrittivo allineato al codice (v1.1)

Build date: 2026-01-24

Obiettivo: descrivere ciò che il progetto HA GIÀ realizzato nel codice e ciò che manca (Gap Register), con tracciabilità verificabile.

Deliverable inclusi nello ZIP

- Documentazione canonica (Overview, PDR, PDD, DataDictionary)
- Traceability Matrix e Parameter Snapshot (allineamento doc-vs-code)
- Evidence Pack (comandi + artefatti attesi) e Gap Register

Indice

Placeholder for table of contents	0
-----------------------------------	---

1. Sintesi esecutiva

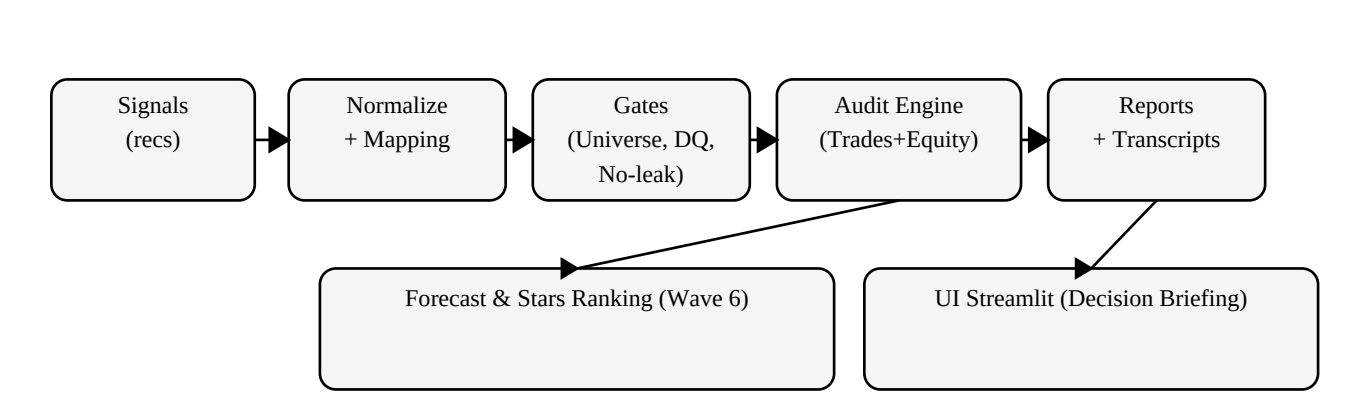
OBSERVER e' un sistema retail-advanced per auditing e certificazione di segnali (incluso news-sentiment): normalizza ticker, controlla universi, evita future leak e genera trade ledger + equity curve, con report e ranking a stelle deterministici.

1.1 Perché esiste

Riduce i tre rischi tipici del retail quantitativo: (i) dati incompleti/instabili, (ii) backtest non riproducibili, (iii) segnali non confrontabili tra fonti.

2. Architettura in breve

L'applicazione e' organizzata in lane operative: SENTINEL-ALPHA (audit/backtest) e NEWS-ALPHA (news ingest/scoring), con data layer DuckDB e UI Streamlit.



2.1 Runner principali (scripts/)

Runner/Comando	Scopo (sintesi)
scripts/news_alpha.py	NEWS-ALPHA operator runner.
scripts/ops_reset.py	Operational reset utility (NEWS-ALPHA / SENTINEL-ALPHA).
scripts/ops_run_session.py	Run -> pack -> certify -> pack (DB-first auto-run-id).
scripts/pack_session.py	Pack reports into deterministic session folders (no symlinks).
scripts/patch_persist_equity.py	Utility operativa
scripts/sentinel.py	SENTINEL-ALPHA one-command runner.
scripts/setup.py	SENTINEL-ALPHA bootstrap / repair entrypoint (cross-platform).

Nota: il runner SENTINEL impone posture offline-by-default, garantendo output riproducibili; azioni online richiedono flag espliciti.

3. Workflow operativo (end-to-end)

Setup → Migrazione schema → Test → Run/Certify → Verify → Forecast Ranking → UI.

3.1 Contratti di progettazione

Offline-by-default: nessun network senza esplicita abilitazione.
Auditability first: run_id, transcript, fingerprint del codice.

No future leak: timing conservativo (T+1).
Retail realism: costi e fiscalità (Italia) incorporati nel backtest.

4. Forecast & Stars Ranking (Wave 6)

Il ranking traduce segnali eterogenei in una graduatoria leggibile (stelle) mantenendo trasparenza (confidence) e robustezza (shrinkage).

4.1 Logica

- 1) bucket stats per (firm,rating) su audit_trades
- 2) shrinkage bucket→firm→global per piccoli campioni
- 3) sentiment adjustment (peso K_SENT)
- 4) confidence = min(1, n_bucket/N_CONF)
- 5) stars per percentile con tie-break deterministici

Parametri principali: N_MIN=20, N_CONF=60, W_GLOBAL=0.25, K_SENT=0.20.

5. Data model (DuckDB)

DuckDB e' il single source of truth locale per riproducibilità: segnali, prezzi, audit runs, ledger e cache sentiment.

5.1 Tabelle principali (scopo)

Tabella DuckDB	Scopo
metadata	Anagrafica strumenti: ticker canonico e attributi di mercato/settore, inclusi flag fiscali (Tobin/FTT) e simboli p
prices	Prezzi giornalieri (barre daily): close (obbligatorio) + open opzionale, con provenance e timestamp di inserim
dividends	Eventi dividendo per realismo retail (ex-date e pay-date). Nello snapshot lo schema e' presente; l'applicazio
recs	Store segnali/raccomandazioni (news/analyst): per data di pubblicazione, ticker, firm e rating, con sentiment
momentum_rankings	Ranking momentum giornaliero per ticker (feature/gate) con rendimento periodo e segnali discreti.
universes	Catalogo universi (es. ALL/US/EU) per filtrare segnali e controllare survivorship bias.
universe_membership	Membership storica (ticker in universo) con intervalli temporali: essenziale per backtest senza survivorship b
ticker_mappings	Mappature alias->canonico time-bounded (corporate actions, cambio simbolo) per evitare mismatch ticker.
ticker_halts	Halt su singolo ticker: finestra temporale di non-tradabilita' e reason/provenance.
market_halts	Halt di mercato/paese (es. circuit breaker, chiusure straordinarie) per execution feasibility.
sentiment_cache	Cache locale deterministica del sentiment (testo normalizzato + hash) per ripetibilita' e audit.
data_gaps	Log operativo di gap/backfill (prezzi/news): include intervalli richiesti, esito, righe inserite e reason_code sta
audit_runs	Registro run di audit/certificazione: configurazione, universi, fingerprint del codice e stato esecuzione.
audit_signal_decisions	Decision ledger opzionale: traccia segnali eseguiti o scartati (dedup, halt, skip) e date 'intese' vs effettive.
audit_trades	Trade ledger per run: ogni trade deriva da un segnale, include shift esecutivi, motivi di uscita e performance
audit_equity	Equity curve giornaliera per run: equity, cash, investito, numero posizioni e tasse pagate.

6. Completezza e verificabilità

Questo documento non si basa su descrizioni generiche: la completezza è garantita da tre ancore:

- **Traceability Matrix:** ogni componente rilevante ha una referenza documentale e un comando di verifica.
- **Parameter Snapshot:** i numeri critici sono estratti dal codice, non riscritti a mano.

- **Gap Register:** ciò che manca è esplicitato e non viene spacciato come implementato.

7. Roadmap retail (monetaria/finanziaria) – suggerita

Step successivi coerenti con un retail disciplinato: macro context layer (tassi/inflazione/regimi), FX translation multi-valuta, dividendi/azioni societarie total return, execution realism dinamico.

Appendice A. Comandi chiave

Setup e migrazione:

```
py scripts/sentinel.py migrate
py scripts/sentinel.py status
py -m pytest
```

Run / Certify / Verify:

```
py scripts/sentinel.py run
py scripts/sentinel.py certify
py scripts/sentinel.py verify --run-id <run_id>
```

NEWS-ALPHA:

```
py scripts/news_alpha.py status
py scripts/news_alpha.py collect --online --allow-online ...
py scripts/news_alpha.py run --fixtures ...
```

UI:

```
py -m streamlit run app.py
```